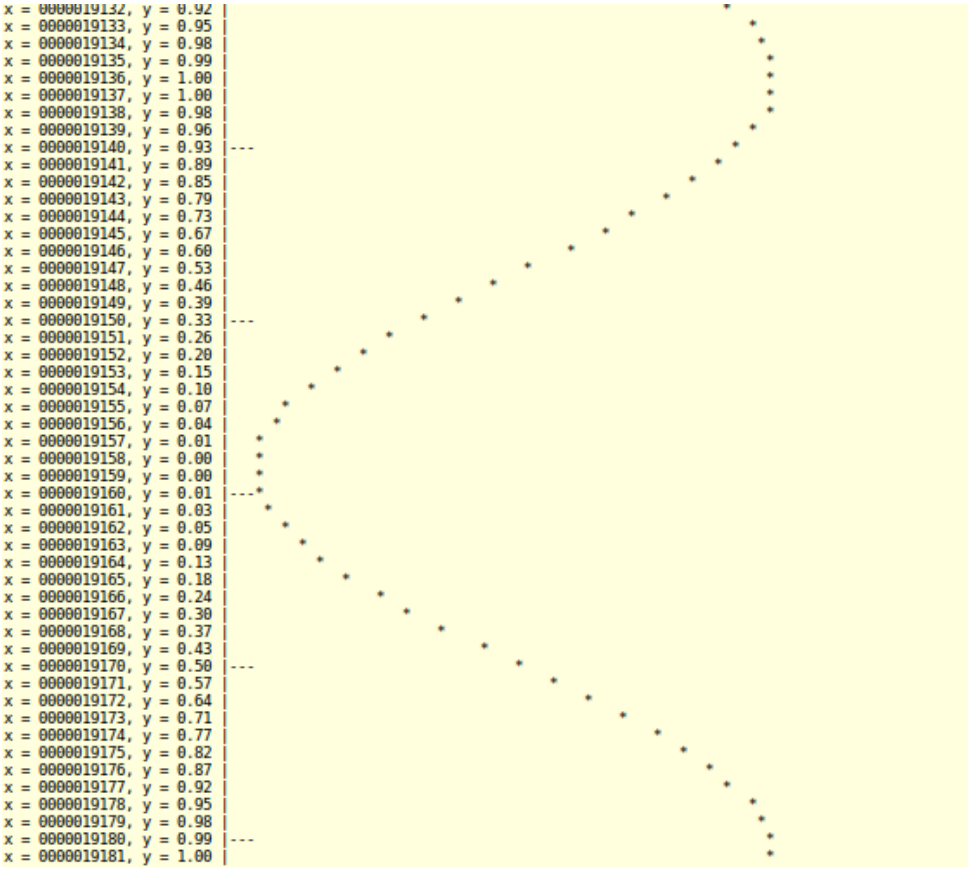
C2

Functions and Control Flow

In this exercise you will build a function plotter in three stages. It will reinforce what you have learned in the first lab, introduce you to calculations using the math library, to writing your own functions with local variables, and to using conditions and loops to control the flow of your code.



Schedule

Preparation Time	:	3 hour
Lab Time	:	3 hours

Items required

Tools	:	
Components	:	
Equipment	:	
Software	:	`gcc`, `notepad++` or equivalent text editor

Version: October 9, 2024

©2012

Klaus-Peter Zauner
Electronics and Computer Science
University of Southampton

1 Introduction

This lab introduces you to programming in C.

1.1 Outcomes

At the end of the exercise you should be able to:

- Write a C program from scratch.
- Write your own functions.
- Use conditions and loops in the design of your programs.
- Perform calculations with formatted output.

2 Preparation

Read the laboratory work section below, read up on any new material (in the Textbook and/or online) you require to accomplish the tasks and make notes in your logbook including the source from where you got the information, so that you can quickly go there again if something does not work as expected. For this lab you will not receive skeleton code, so you should be prepared to open an editor and write a C-program from scratch, based on what you learned in the first lab.

At least you should have the following in your logbook before the start of the lab:

- Notes about declaring an unsigned long variable, how to initialize it correctly, and what format specifier is required to print such a data type.
- Notes on how to print a number with a fixed character width.
- Notes on how to create an endless loop in C.
- A formula that gives you the sine of a value scaled to the range of 0 to 1. (You should also have an idea how to change the frequency in your formula.)
- A draft of the `plotval()` function needed in Part 2. Consider to use a loop to print an appropriate number of spaces into the same line.

3 Laboratory Work

3.1 Part 1

Write a C-program that in its `main()` function declares a local unsigned long variable named `x` and initializes the variable with a value of zero. After initializing `x` the program should enter an endless

loop. Within the loop the program should increment `x` (i.e., add one to x) and print the value of `x` with a fixed width.

Compile and start your program on the command line. You can stop your program[1] by typing Ctrl-C in the terminal.

Debug your program and make sure it easy to read for humans (correct indentation, comments), then paste it into your logbook[2]. 

Make a copy of the code you have developed so far. Include the math library (`math.h`). Note, that you will need to let the compiler know that it should link with the math library, by using the option `-lm` before the output option.

Declare in `main()` a variable named `y` of type float. In your endless loop, modify the printing of `x` such that it does *not* end the line. Following the printing of `x`, calculate the value of `y` as the sine function of `x` multiplied by $\frac{\pi}{180}$. Define `PI` with the preprocessor. After the calculation of `y`, print it with fixed width and two or three digits after the point. Let this end the line.

When this works, modify the formula such that the value of `y` is shifted and scaled to fall into the range 0...1.

Verify that the scaling works, then define a constant FREQ that can be used in your formula to increase the frequency of the sine function (assuming x is a time), you can set it to 1 and adjust it later.

Debug your program and make sure it is easy to read for humans, then paste it into your logbook. 

3.2 Part 2

In this section you will visualize the function calculated in the previous part. Make a copy of your code from Part 1.

Declare and define a function `plotval()` that takes two arguments and returns nothing. The first argument is a floating point value in the range between 0 and 1, and the second argument is an integer, the width of the terminal that is available for plotting. The latter is given in characters, and will depend on the width of the terminal available. Define a constant for the width of the plot and use it in calls to the `plotval()` function as second argument; a good value may between 50 and 60, depending on how you have formatted your numbers in Parts 1.

The function `plotval()` should print a symbol (e.g. '*') at the current position in the line if the first argument is 0, plot it shifted to the right by the number of characters given in the second argument, if the first argument is 1, and position the symbol between these extremes proportionally to the value of the first argument, if it is between 0 and 1. This function should also end the line.

Debug your program and make sure it is easy to read for humans, then paste it into your logbook. 

[1] As well as most other programs that have been started on the command line.

[2] In case you use a paper logbook and do not have a printer, insert a picture of your code (e.g. a screenshot from the editor) in the set of images you make from your logbook for the electronic submission.

3.3 Part 3

Make a copy of your code from Part 2.

Modify your main loop such that it shows a tick-mark following the numbers at regular intervals (The cover page shows a tick mark at every ten values). Use an `if...else` condition together with the modulo-operator `'%'` to print either the mark or an equivalent number of spaces. Define a constant that can be used to change the spacing of the tick-marks.

Debug your program and make sure it is easy to read for humans, then paste it into your logbook.

Improve the output formatting to your liking, paste an illustrative piece of the curve into your logbook.

Consider what determines the speed with which your program executes; make a note about your thoughts.

4 Optional Additional Work

Plot more interesting periodic functions, e.g. summations of sine functions with different phase and frequency. You can add noise with the `rand()` function from the math library. Paste a representative part of an interesting curve into your logbook and write the formula you used for this in your logbook.

Use Fourier Synthesis (see https://en.wikipedia.org/wiki/Fourier_series) to add a number of suitable sine waves together to produce an approximation of a square wave. Allow in your program that the number of harmonics which are added, can be varied (either by user input or as constant set at compile time).

Paste a few graphs into your logbook that show how the approximation of the square wave improves with an increasing number of harmonics; see also the animation at the end of https://en.wikipedia.org/wiki/Square_wave.

Make sure you cite all sources of information you have used in the comments of your code and in your logbook.